PROGRAMMING LANGUAGE CONCEPTS

2.OOPS (Object-Oriented Programming System)

🔷 OOPS (ऑब्जेक्ट ओरिएंटेड प्रोग्रामिंग सिस्टम) क्या है?

OOPS एक प्रोग्रामिंग पैरेडाइम (शैली) है जिसमें हम **ऑब्जेक्ट्स** और **क्लासेस** की मदद से प्रोग्राम बनाते हैं। इसका मुख्य उद्देश्य है:

- **कोड को मॉड्यूलर बनाना**
- **रियूजेबिलिटी बढ़ाना**
- **डाटा को सुरक्षित रखना (Encapsulation)**
- **जटिलता को छुपाना (Abstraction)**

🧩 OOPS के 4 मुख्य सिद्धांत (Main Concepts):

1. Class (क्लास)

- **परिभाषा:** क्लास एक **ब्लूप्रिंट (design)** होती है जिसमें हम ऑब्जेक्ट्स के प्रॉपर्टीज (गुण) और मेथड्स (क्रियाएं) को define करते हैं।
- **उदाहरण:** "Car" एक class हो सकती है, जिसमें color, brand, speed जैसे attributes हों।
- **कोड उदाहरण:**

```
class Car:
    def __init__(self, brand, color):
        self.brand = brand
        self.color = color
```

---

2. Object (ऑब्जेक्ट)

- **परिभाषा:** Object क्लास का **रियल वर्ल्ड इम्प्लीमेंटेशन (instance)** होता है।
- हर ऑब्जेक्ट के पास अपनी **properties (state)** और **behaviors (क्रियाएं)** होती हैं।
- **उदाहरण:** BMW और Audi – ये दोनों "Car" class के object हो सकते हैं।
- **कोड उदाहरण:**

```
car1 = Car("BMW", "Black")
car2 = Car("Audi", "White")
```

---

3. Encapsulation (इनकैप्सुलेशन) – डेटा को सुरक्षित बनाना

- **परिभाषा:** जब हम **डाटा (variables)** और **फंक्शन्स (methods)** को एक ही यूनिट में bind करते हैं, और बाहरी दुनिया से **हाइड (छुपाते)** हैं, तो उसे Encapsulation कहते हैं।
- **लाभ:** डाटा सेफ रहता है और कोड कंट्रोल्ड रहता है।
- **कोड उदाहरण:**

```
class Person:
    def __init__(self, name, age):
        self.__name = name  # private variable
        self.__age = age

    def get_age(self):
        return self.__age
```

---

4. Abstraction (एब्स्ट्रैक्शन) – जटिलता को छुपाना

- **परिभाषा:** Abstraction का मतलब है कि यूज़र को सिर्फ जरूरी जानकारी मिले, और बैकग्राउंड की जटिलता छुपी रहे।
- **वास्तविक जीवन उदाहरण:** आप मोबाइल चलाते हैं, लेकिन आपको ये नहीं पता होता कि उसके अंदर का सॉफ्टवेयर कैसे काम करता है।
- **कोड उदाहरण (Python में):**

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass

class Dog(Animal):
    def make_sound(self):
```

```
        print("Bark")
```

---

5. Inheritance (इनहेरिटेंस) – विरासत में लेना

- **परिभाषा:** जब एक class दूसरी class के गुण (properties) और व्यवहार (methods) को **विरासत में लेती है**, तो उसे Inheritance कहते हैं।
- **लाभ:** कोड रियूजेबल हो जाता है।
- **उदाहरण:** एक "Animal" class से "Dog" class को गुण मिल सकते हैं।
- **कोड उदाहरण:**

```
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    def bark(self):
        print("Dog barks")

d = Dog()
d.speak()  # inherited method
d.bark()
```

---

6. Polymorphism (पॉलीमॉर्फिज्म) – एक नाम, कई रूप

- **परिभाषा:** जब एक ही नाम का function या method अलग-अलग तरीकों से behave करता है, उसे Polymorphism कहते हैं।
- **उदाहरण:** एक ही method `sound()` अलग-अलग animals के लिए अलग-अलग आवाज़ निकाल सकती है।
- **कोड उदाहरण:**

```
class Bird:
    def sound(self):
        print("Bird makes sound")

class Sparrow(Bird):
    def sound(self):
```

```
        print("Chirp Chirp")

class Crow(Bird):
    def sound(self):
        print("Caw Caw")

for bird in (Sparrow(), Crow()):
    bird.sound()
```

📋 **सारांश (Summary):**

सिद्धांत (Concept)	**विवरण (Explanation in Hindi)**
Class	ऑब्जेक्ट का खाका (ब्लूप्रिंट)
Object	क्लास का असली उपयोग या instance
Encapsulation	डेटा को छुपाकर सुरक्षित करना
Abstraction	जटिलता को छुपाकर आसान इंटरफेस देना
Inheritance	एक क्लास से दूसरी क्लास में गुण लेना
Polymorphism	एक ही नाम का method, अलग-अलग behavior

OOPS के फायदे (Advantages of OOPS)

- ✅ Modular Code (आसान मेंटेनेंस)
- ✅ Code Reusability (बार-बार कोड नहीं लिखना)
- ✅ Scalability (बड़ा प्रोजेक्ट आसान बनता है)
- ✅ Real-world modelling (जैसे बैंक, स्कूल सिस्टम आदि)
